

ParallelBayes: 时间并行 MCMC 的可核验实现与成本评估

中文研究稿: 作者与单位待定

2026 年 10 月

摘要

时间并行 MCMC 同时求解多个采样位置, 尝试缩短单条轨迹的计算时间, 其收益取决于并行计算能否抵消额外求值和同步。本文介绍带 R 接口的研究软件 ParallelBayes, 将目标函数、转移核和执行器分开实现, 并在固定实际随机输入下核对路径、接受事件和成本。在所测 M4 CPU 配置中, 两类时间执行器均未取得组中位缓存执行加速; 在独立的 Windows 实验中, RTX 5080 上 Picard 的组中位缓存速度比为 1.10–5.23, quasi-DEER 仍较慢。部分 Picard 收益伴随全部拒绝, 说明较短执行时间可与无效探索同时出现。已有机制对照显示, 窗口宽度带来的批处理机会会被重复求值和较慢的前缀推进抵消。本文据此区分同核执行成本与后验函数误差; 采用独立调参与分散初值的固定预算推断研究正在进行, 其结果尚未计入本文。

1 引言

MCMC 以相邻状态的递推依赖换取对复杂后验的可计算访问。多链并行同时运行多条轨迹; 时间并行则尝试同时求解同一条轨迹中的多个位置。Newton 类路径求解与增量 Picard 提供了两条具体路线 [1, 2], 但并行求值还会引入重复工作、导数近似和同步, 其局部执行收益是否保留到后验推断, 需要区别考察。

本文围绕两个问题展开: 固定核参数、初值及实际随机输入时, 时间执行能否更快完成同一递推; 允许不同核采用各自配置后, 所选后验函数的误差与 workflow 成本如何变化。第一类比较控制轨迹的统计信息, 第二类同时受初始化、调参和混合影响。本文实现 ParallelBayes, 为这两类比较提供可复用目标接口、独立路径参考、实际输入与失败记录, 并用指定 CPU/CUDA 条件下的实验解释成本差异。

模型访问与推断组件复用 Stan、BridgeStan、BlackJAX 和 Pyro[3, 4, 5, 6]。compareMCMCs、PPL Bench 与 posteriordb 已有跨引擎运行、基准或参考目标支持 [7, 8, 9]; ParallelMCMC.jl 和 BayesForge 分别覆盖时间并行 MALA 接口与 R/JAX 建模路线 [11, 12]。本文增加的是同一实际输入下的执行比较和数值核验流程, 以及这些流程揭示

的成本与探索差异，不以统一 R 接口本身作为创新依据。研究限于固定维度连续参数、两类 MH 时间执行和成熟 NUTS 基线。

2 方法与软件设计

2.1 目标与坐标

设 T 为相应自由坐标间的可微一一变换，原参数为 $\theta = T(q)$ ，计算使用无约束坐标 $q \in \mathbb{R}^d$ ，目标为

$$\ell(q) = \log p(y, T(q)) + \log |\det J_T(q)|. \quad (1)$$

模型记录参数顺序、约束映射、完整数据、实现版本与指纹。两个提供方式可省略不同的参数无关常数，因此以 $\ell(q) - \ell(q_0)$ 、同坐标梯度及变换结果核对目标。正值测试例使用 $\theta = \exp(q)$ 和 lognormal 密度，其 $-\log \theta$ 项与 Jacobian 抵消。这一例同时用于检测遗漏 Jacobian 的实现错误。

R 是建模控制与分析入口。BridgeStan 在 CPU 上调用编译后的 C++ 密度和自动微分，Python 串行循环组织转移；原生 JAX 模型进入 JIT 和批处理；独立 torch 后端使用 NumPy 参考及显式 CPU/CUDA 执行，当前时间执行器仍含 Python 主机控制。Stan 模型指针不跨线程共享，Stan 内部线程关闭。此时高频调用边界的成本属于软件工作流，不能从跨后端差异直接推断并行算法的贡献。

目标入口与实测设备	顺序 RWM/MALA	Picard RWM / quasi-DEER MALA	NUTS
Stan/BridgeStan, Mac CPU	支持	不支持	不支持
明确 JAX 目标, CPU	支持	支持对应核	BlackJAX 顺序核
明确 torch 目标, Windows CPU	已核验	已核验, 主机控制	独立研究 CLI: Pyro 四链
明确 torch 目标, Windows CUDA	已核验	已核验, 主机控制	未接入/核验

表 1: 当前代码的支持范围与实测边界。CPU Pyro 基线是独立研究 CLI，不是 `pb_sample()` 的通用 torch NUTS 选项；四链并发也不是 NUTS 轨迹时间并行。原生 Windows Stan 编译尚未核验。支持数值输出不等于已证实后验精度。

R 函数 `pb_capabilities()` 查询包接口支持的组合；独立研究 CLI 另列运行契约，不能从表 1 推定全部路线都能由一个 R 函数调用。`pb_benchmark()` 只组织配置列表及 R 结果，正式协议冻结、检查点和完整证据管理依赖 Python CLI 及项目文件。Stan 模型没有默认独立轨迹参考，`audit=TRUE` 明确拒绝。扩展示例通过公开 Python Model 提

供 Poisson 目标及独立 NumPy 参考，R 经 reticulate 调用并转换为 posterior 对象；并非 `pb_model()` 可直接接收任意 R 闭包。

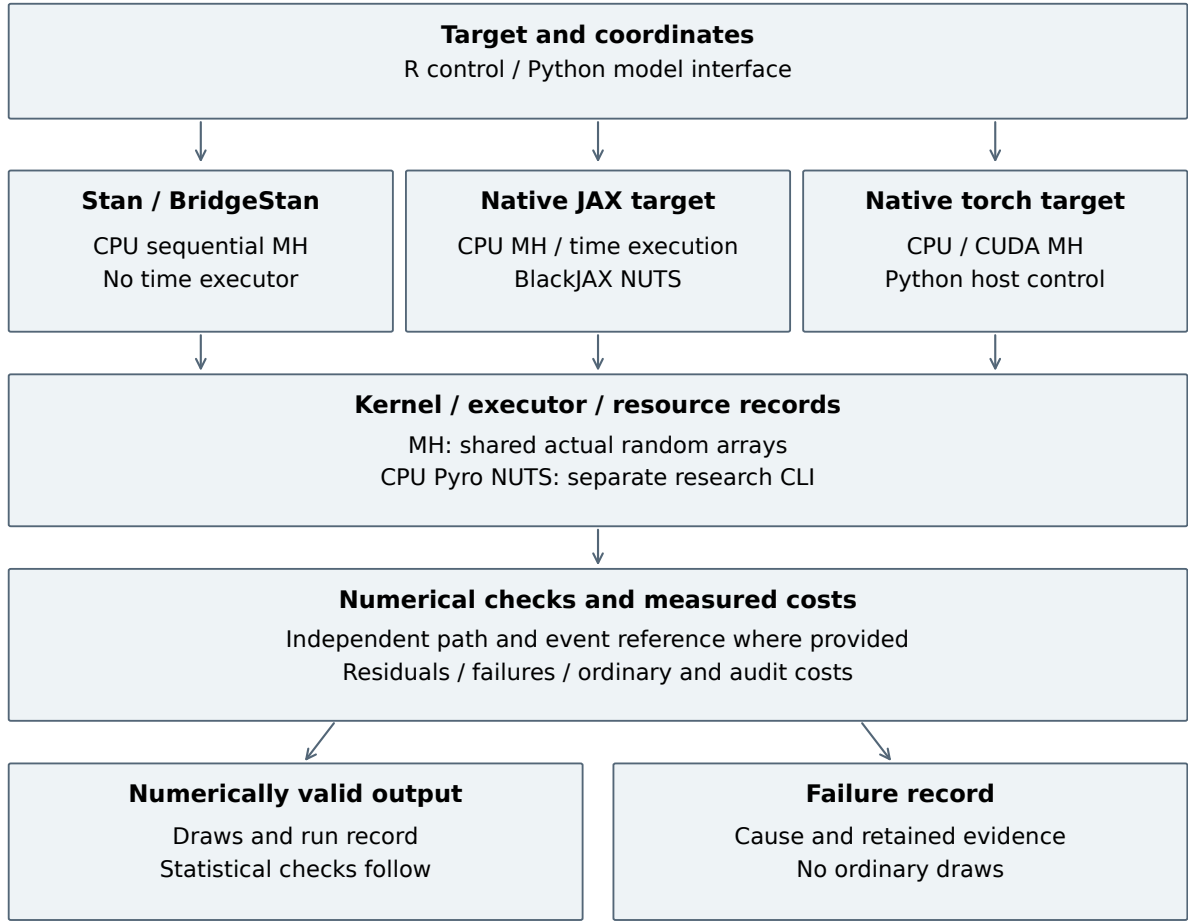


图 1: 当前软件路线与证据流。Stan 只连接 CPU 顺序核；JAX 和 torch 仅接受明确实现的目标。独立顺序参考须另外提供，Stan 入口没有默认参考。NUTS 使用自身随机流及预热，独立 Pyro CLI 另作记录。数值有效输出继续接受统计诊断，失败不能转为正常样本。

2.2 固定随机输入下的核

MALA 的步长约定为

$$q' = q + h\nabla\ell(q) + \sqrt{2h}z, \quad z \sim N(0, I_d). \quad (2)$$

接受对数比包含正反向提议密度：

$$a(q, q') = \ell(q') - \ell(q) - \frac{\|q - q' - h\nabla\ell(q')\|^2 - \|q' - q - h\nabla\ell(q)\|^2}{4h}. \quad (3)$$

当 $\log u < a(q, q')$ 时接受，否则保留 q 。随机游走 Metropolis (RWM) 使用 $q' = q + sz$ ，接受对数比为 $\ell(q') - \ell(q)$ 。拒绝产生的自环计为一个转移。本文的 h 不等于提议噪声标准差；这一约定在所有配对执行中固定。

随机输入由锁定 NumPy 版本下的 Philox 生成，采样与导数探测种子分开。比较固定的是噪声数组和对数均匀随机数组，而非跨软件的同名整数 seed。正式原始输出保存这些数组及其校验和。独立顺序参考使用 NumPy 密度、解析梯度和显式循环，不调用待核验的 JAX 或 torch 转移函数。跨系统重建派生浮点数组不保证逐位一致；重复执行应传递已封存的实际数组，再分别核对文件与数组哈希。

2.3 两类时间执行的共同数学对象

普通 MCMC 逐步生成状态。时间执行器先猜测一个窗口内的路径，再修正相邻状态的不一致。固定初值和实际随机输入后，每一步都是确定映射，顺序路径满足

$$q_t^* = F_t(q_{t-1}^*), \quad t = 1, \dots, W. \quad (4)$$

quasi-DEER 用近似导数修正窗口；Online Picard 用接受增量重建路径并提交连续一致的前缀。它们求解的是指定递推，核验比较的也必须是相同初值、核参数及随机数组。

quasi-DEER 的窗口修正。 给定第 k 轮路径，下一轮满足

$$q_t^{(k+1)} = F_t(q_{t-1}^{(k)}) + B_t^{(k)} \{q_{t-1}^{(k+1)} - q_{t-1}^{(k)}\}. \quad (5)$$

旧路径上的映射和矩阵可同时计算；随后把右侧整理为关于新前驱的仿射递推，用结合扫描求解。在精确算术、各次映射有定义、 $B_t^{(k)}$ 有限且仿射递推精确求解时，第 k 轮至少恢复前 k 个状态，因而长度 W 的窗口至多需 W 轮。原因是固定初值使第一步正确；若前 k 步已经正确，下一轮保持该前缀，并使第 $k+1$ 步的修正项为零。这是既有因果递推性质 [16, 1]，不依赖 B 必须为真实 Jacobian。它没有为浮点溢出、容差提前停止或一般后验误差提供保证。

硬前向与代理 Jacobian。 令 $b = \mathbf{1}\{\log u < a(q, q')\}$ 、sg 为停止梯度算子。本实现用

$$g = b + \sigma(a - \log u) - \text{sg}\{\sigma(a - \log u)\} \quad (6)$$

构造代理导数；前向仍返回硬接受/拒绝状态。torch 代码另以停止梯度包裹硬输出，使前向避免软表达式的舍入抵消。这里得到的是代理 Jacobian J_{proxy} ，不是硬 MH 映射的真实 Jacobian。每链、每时间位置采用一个实际 Rademacher 向量 v_t ，在窗口迭代中复用，并设置

$$B_t^{(k)} = \text{Diag}\left[\text{clip}\{v_t \odot (J_{\text{proxy},t}^{(k)} v_t), -1, 1\}\right]. \quad (7)$$

截断作用于导数近似，不作用于状态。固定探针的复用不支持额外声称每轮自适应矩阵的条件无偏估计。

原论文伪代码先取 $\log \alpha = \min(0, a)$ ，其平滑门控为 $\sigma(\min(0, a) - \log u)$ [1]；本项目归档提交 5e5b637 的上游 samplers.py 与本实现使用未截断的 a 。两种硬接受判据等

价，代理导数一般不同，故这里明确报告所测公式。本实现还采用非重叠窗口、重复窗口起点的初猜及硬残差停止，不使用上游的 NaN 归零或状态裁剪；性能比较针对该适配实现。

Online Picard 的增量和确认前缀。 已确认起点为 q_L 时，RWM 的固定输入增量及候选路径为

$$D_t^{(k)} = sz_t \mathbf{1} \left\{ \log u_t < \ell(q_{t-1}^{(k)} + sz_t) - \ell(q_{t-1}^{(k)}) \right\}, \quad (8)$$

$$q_t^{(k+1)} = q_L + \sum_{i=L+1}^t D_i^{(k)}. \quad (9)$$

在新路径的前驱上重算接受事件，只提交连续匹配的前缀。例如四个位置依次为“匹配、匹配、不匹配、匹配”，只能提交前两个。第四个位置的前驱尚未确认。由起点向后归纳，固定噪声下匹配的接受事件确定相同增量，因而确认前缀满足原递推 [2]。torch 批量执行对各链取最短确认长度共同推进，窗口尾部只处理实际存在的转移。浮点前缀和与顺序累加的次序不同，仍须分别检查路径差和接受事件。

2.4 数值输出标准与失败处理

quasi-DEER 停止条件为硬递推残差

$$r = \max_{t,j} \frac{|q_{t,j} - F_t(q_{t-1})_j|}{\text{atol} + \text{rtol}|F_t(q_{t-1})_j|} \leq 1. \quad (10)$$

它控制局部一致性，而非长路径误差。独立 NumPy 参考另要求接受事件零失配，且每链无约束路径最大差不超过 $100[\text{atol} + \text{rtol} \max(1, \max |q|)]$ ，本文协议的两种容差均为 10^{-10} 。100 倍系数是指定输出标准，不是误差传播界。全路径尺度会放宽小尺度坐标的阈值；后续逐坐标和函数差只能作为另列的描述，不能追溯改变原合格判据。

迭代耗尽、非有限值或独立审计失配的输出保留原记录，不转换为普通后验样本。当前 MH 实现把非有限提议密度统一视为失败；合法的支持集外提议密度 $-\infty$ 尚未一般地区分，因此有界支持目标的适用范围受限。NUTS 的发散与返回有限数组分别记录。显式顺序回退复用原数组重算整个轨迹并计入失败费用，本文实验不启用回退隐藏求解失败。

2.5 R 用户如何运行和扩展

安装版示例以二维 Poisson 回归为例， $y_i \sim \text{Poisson}(\exp(X_i\beta))$ 、 $\beta_j \sim N(0, 4)$ 。外部 Python 模型提供框架密度、独立 NumPy 密度/解析梯度和恒等参数变换；R 整批调用，再转换为 posterior 对象。以下为仓库中实际示例的调用顺序，环境变量在启动 R 前指定，输出使用不存在的新目录。

```
# Once: create the shared data and actual random arrays
python examples/installed_custom_target.py prepare \
  --output inputs
# Set R_LIBS_USER and RETICULATE_PYTHON before starting R
Rscript --vanilla examples/installed-custom-target.R \
  torch inputs output cpu
```

R 脚本核心调用如下；`example_dir` 定位随包交付的示例文件，`inputs/output` 为上述目录。

```
library(parallelbayes)
pb_environment("torch")
helper <- reticulate::import_from_path(
  "installed_custom_target", path = example_dir, convert = TRUE)
result <- helper$run_example(inputs, output, "torch", "cpu")
for (w in result$workflows) {
  if (!identical(w$status, "completed")) next
  draws <- posterior::as_draws_array(w$draws)
  print(posterior::summarise_draws(draws))
}
```

完整脚本同时核对安装模块来源和输入哈希，保存失败项、样本及诊断，并在数值检查失败时返回错误。该例已在 Mac 安装环境下运行 JAX/torch 与 R 入口，四链各 128 次转移用于说明接口；没有预热，不能作为推荐推断预算。它的 Windows/CUDA 组合尚未实测。水井案例另有 Windows CPU/CUDA/R 接口证据。`pb_model()` 不接收任意 R 闭包，Pyro CPU NUTS 也仍是独立研究 CLI。

3 核验方法与实验设计

3.1 实验身份、目标与独立单位

各批次回答不同问题，不能按任务数合并统计证据（表2）。同核配对共享初值、核参数和实际数组；链、缓存回放、跨设备副本均不增加独立重复数。历史 Mac 为 M4/16 GB、JAX float64；Windows 为 Ryzen 7 9800X3D、RTX 5080 和 torch float64，主机控制与同步成本保留。平台间同时改变实现和硬件，只能比较完整 workflow。

目标库涵盖独立与相关高斯 G1/G2、具有高斯后验的潜在 AR(1) A1、logistic L1/L2、中心化/非中心化漏斗 H1/H2、双峰 M1 和水井 W1。历史两轮不含 W1。H1/H2 以不同坐标采样，原尺度输出通过变换对应；相同数值步长不代表原尺度上的同一核，因此只在每个坐标表示内部配对执行。

证据批次	平台与 workflow	重复和保留预算	用途/当前状态
Mac 主实验	0.1.0, 五 workflow	每格 24 份四链输入; 256/2048	1920 项, 固定预算比较
Mac 修订/SBC	0.1.1, 解析生成模型	64 数据集; 五共享工 作流	配对解析校准检查
Windows 首轮	0.2.0.dev1, CPU/CUDA 四种 MH	每格 4 份四链输入; 128/512	512 项, 探索性执行比 较
Windows 机制	独立机制协议, CPU/CUDA	每模型 2 份输入; 窗 口 4/16	192 工作流; 机制描述
紧凑推断研究	独立冻结版本; 九工作 流	每目标 24 份四链输 入; 1024/4096	3888 主任务; 运行中、 未纳入结果

表 2: 证据身份与用途。缓存回放和安装/恢复检查不计统计重复; 详细源码、环境及协议清单见补充材料与项目索引。

3.2 函数误差、参考和诊断

对预先规定的函数 f , 记录四链平均估计 $\hat{\mu}_{a,f}^{(r)}(N)$, 逐函数误差为

$$\hat{E}_{a,f}(N) = \frac{1}{R} \sum_{r=1}^R \frac{(\hat{\mu}_{a,f}^{(r)}(N) - \mu_f^*)^2}{s_f^2}. \quad (11)$$

有解析期望时这是重复 MSE 的估计; 有限 MCMC 参考下只能称参考平方差。历史分析还报告函数间最大值, 紧凑研究以逐函数结果为主。所有实际函数定义列于表3; 标准化已包含在部分函数本身, 其余函数没有转成相同实际意义的相对误差。当前误差计算的额外 $s_f = 1$ 。历史阈值 $\epsilon = 0.1$ 即平方误差 $\epsilon^2 = 0.01$, 对应函数单位中的 RMSE 0.1; 它是比较标尺, 不是每种应用都充分的精度要求。误差估计的重复不确定性和参考 MCSE 均另列, 不与 ϵ 混用。

目标 (函数数)	原尺度函数, 所有 $s_f = 1$	期望/参考
G1/G2/A1 (各 3)	$z, z^2, \mathbf{1}(z > 1), z = (\theta_1 - m_1)/\sqrt{\Sigma_{11}}$	0, 1, $1 - \Phi(1)$
L1/L2 (各 4)	$\theta_1, \theta_2, \sigma(\theta_1), \mathbf{1}(\theta_1 > 0)$	独立 NUTS 有限参考; L2 符号 事件未定
H1/H2 (各 4)	$v/3, \tanh(v/3), \mathbf{1}(x_1 > 0), \cos(x_1 e^{-v/2})$	0, 0, $1/2, e^{-1/2}$
M1 (4)	$\theta_1/\sqrt{1+\delta^2}, \mathbf{1}(\theta_1 > 0), \theta_2, \theta_2^2$	0, $1/2, 0, 1$; δ 为模式位置绝对值
W1 (7)	$\alpha, \beta, \alpha^2, \beta^2, \sigma(\alpha), \sigma(\alpha + \beta), \mathbf{1}(\beta > 0)$	非认证数值求积及独立有限参 考; 非解析真值

表 3: 九目标共 36 个函数; W1 两项预测概率对应 0 米和 100 米。完整数值参考与不确定性由固定参考契约提供。历史 $s_f = 1$ 口径保留, 不用正式样本方差重新归一化。

L2 参考的符号事件在 262144 个相关样本中恒零，不能据零经验方差填入零误差。W1 后验可正规化的证明不认证求积尾部或稀有事件精度。对有限参考的敏感性分析与条件于参考的统计区间分开给出。现代秩正态化/折叠 $\hat{R}$ 、bulk ESS 与 tail ESS 从完整样本计算 [17, 10]，常量输出保留不可判定；这些诊断不替代真实函数误差。

历史 Mac 采用 24 重复、4000 次逐点 bootstrap；原 Windows 每格只有 4 份输入，其 bootstrap 区间只作探索性描述，正文直接展示配对点。紧凑研究主任务按完整四链重复整体重采样，采用 9999 次逐点 BCa、至少 20 份共同可用重复；退化区间保留未定。缓存每格仅 4 份输入，不计算 BCa。失败率使用全部计划重复，误差/费用的成功条件统计明确标注；它们不等于包含失败损失的无条件风险。

3.3 调参、预算与三种成本边界

历史零初值和固定 MH 步长定义的是指定工作流。新紧凑研究沿用独立开发试验选定的仿射坐标和步长：每目标/核六个候选，每候选两份四链输入，按两次都合格后的平均 ESJD/d 选择。216 次计划开发运行中 212 次完成、4 次失败；该规则优化移动距离，不证明 ESS 或函数误差最优。正式 MH 初值在采样坐标中取 $N(0, 4I)$ ，M1 首坐标分置两侧；窗口 32，quasi-DEER 每窗口最多 2048 轮，Picard 最多 $N + 512$ 轮。CPU Pyro NUTS 使用四进程，每链预热 1024 步、目标接受率 0.8、最大树深 8；MH 舍弃 512 步。开发调参费用单列，不能隐记为零或假定每次拟合重新支付。

紧凑研究固定两档保留预算 1024/4096 及 24 重复。该设计因用户硬件与空间条件在旧大网格启动后修订，保留旧证据并用新身份和输入，不能称完全事前预注册。现阶段按用户进度报告正在运行，实际冻结源码和全部结果待独立接收。本稿不加入尚未回传的误差或加速结论，也不把有限预算曲线插值成精确达标时间。

计时分别回答三个问题：缓存执行衡量准备后的内核；普通批量 workflow 包含进程启动、输入/目标、适应或丢弃、采样、变换、输出和 R 诊断；研究审计另含独立顺序参考与完整证据检查。CUDA 边界显式同步，后两种口径为嵌套边界，不相加。历史 Mac 的普通内存口径只是从已有计时扣除审计的记账估计；后继进程测量才是直接观测。两者仍不含最外层 R 前端、环境安装和全局归档传输；R 完整调用须另批测量，不能回填历史主任务。误差与时间使用同一组函数可用重复，缺少完整计时的任务不能通过删行产生曲线。

4 实验结果

4.1 路径一致性与统计校准是不同检查

Mac 的 1920 项和 Windows 首轮的 512 项均通过各自规定的数值输出检查，独立参考的接受事件零失配；最大路径差分别为 2.36×10^{-8} 和 5.212×10^{-10} 。两者容差尺度与实现不同，不能用这两个最大值作算法精度排名。跨系统诊断重建还发现小数读取

与中位数末位变化影响秩诊断；改用实际二进制输入后，剩余差异定位于折叠分量，未改变相关任务诊断不良的判断。详细定位及失败记录进入补充材料。

SBC 采用 $\theta \sim N(0, 1)$ 、 $y_j \mid \theta \sim N(\theta, 1)$, $j = 1, \dots, 8$, 解析后验为 $N(\sum_j y_j/9, 1/9)$ 。在同一批 64 数据集上，解析 90% 区间覆盖 54 例；MALA 及其时间执行也覆盖 54 例，但与解析区间有 4 例覆盖分歧。RWM 及 NUTS 覆盖 53 例。总体比例接近不能说明逐数据集区间相同，五种工作流也不能算五份独立校准证据。端点、后验矩和数据依赖对数似然秩另列；测试量的选择决定 SBC 能发现哪些问题 [18]。

4.2 指定轨迹的执行成本

表4汇总历史模型—预算组的缓存速度比。Mac 两种执行器均慢于配对顺序实现；Windows Picard 在 CUDA 的各组中位数均大于 1，quasi-DEER 仍全部较慢。这些是同设备、同后端、同核比较，不能解释为 GPU 相对 CPU 的统一加速。

历史实验/设备	quasi-DEER MALA	Picard RWM
Mac / JAX CPU	0.0192–0.174	0.0503–0.809
Windows / torch CPU	0.101–0.359	0.774–3.560
Windows / torch CUDA	0.164–0.505	1.097–5.230

表 4: 各模型—预算组配对缓存速度比中位数的范围；顺序时间/时间执行器时间。Mac 每格 24 份输入，Windows 每格 4 份；技术重放不增加重复。

Mac 研究审计工作流的组中位速度比为 quasi-DEER 0.0715–0.739、Picard 0.226–0.814。加入共同审计成本会使比值趋近 1，可能掩盖加速，也可能掩盖减速。由于缓存执行本身已经较慢，Mac 负结果不能只归因于独立核验费用。

4.3 为什么较快的执行未必提供更有用的样本

CUDA 的 A1/RWM、512 步配置中，Picard 缓存执行约快 4.09 倍，但提议全部被拒绝，链停在初值，执行器每轮可确认整个窗口。自环降低了路径求解难度，同时使后验探索失效。这一反例解释了确认前缀与统计效率的区别，不能只按每次确认转移的成本推荐执行器。

窗口扩大还会增加重复工作。Mac 机制实验中，G1 四链 Picard 将窗口 16 增至 64，每轮确认数的组中位数仍约 5.57，而前向映射次数由 5.50 升至 20.88 次/输出转移，缓存速度比从 0.402 降至 0.170。G2 单链 quasi-DEER 的平均窗口轮数由 11 升至 22.25，速度比从 0.051 降至 0.027。拒绝自环均计入分母，JVP 另计。

Windows 机制协议也包含 G2 RWM 步长对照、窗口 4/16 及链数分配。quasi-DEER 每输出转移需 7.75–22.5 次前向映射及 3.875–11.25 次 JVP；Picard 需 2–7.59375 次前向映射。部分零接受配置一次确认整个窗口。等 512 总输出的另八个对照中，16 条短

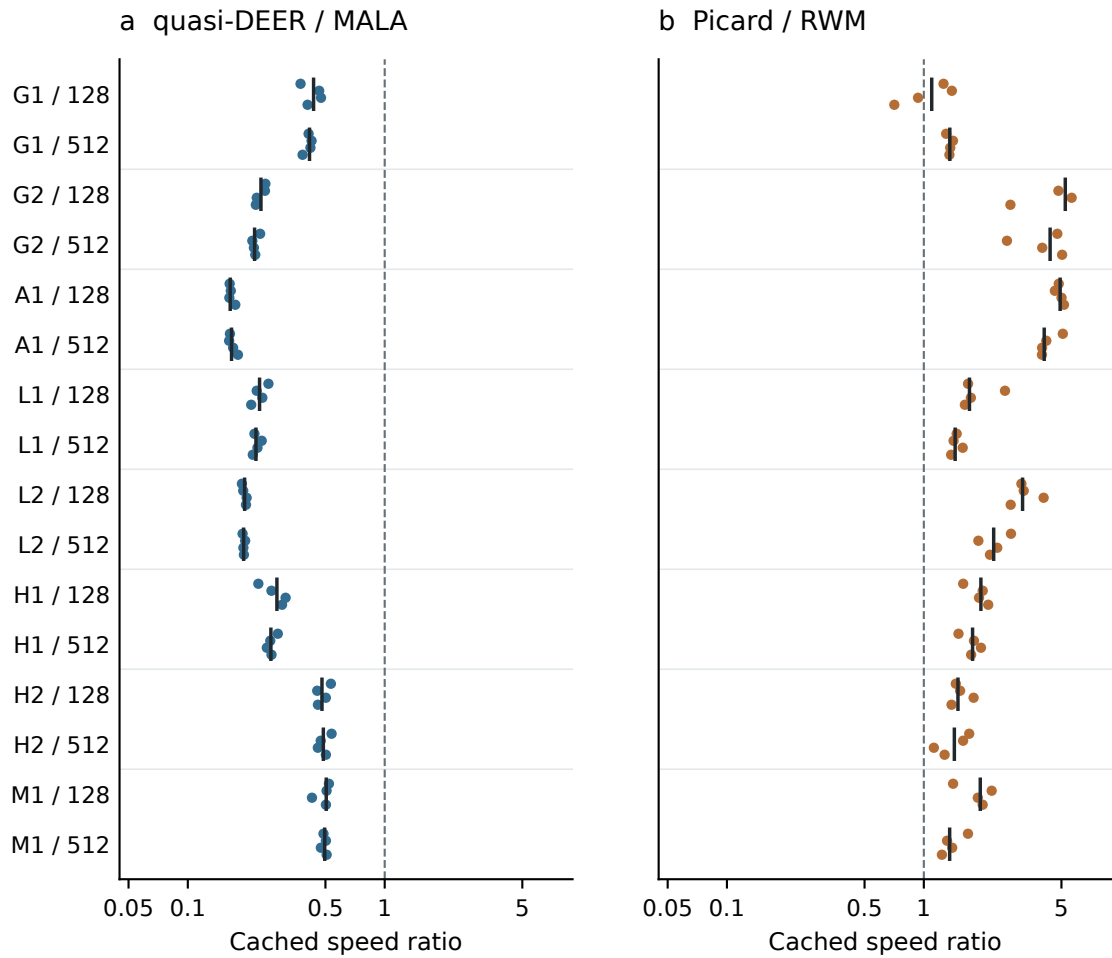


图 2: 首轮 Windows CUDA 的全部配对点。每行一个模型与预算，四个点对应四份实际输入，黑色短线为中位数；虚线为速度比 1。左为 MALA/quasi-DEER，右为 RWM/Picard。逐点展示替代小样本区间的强性能解读；CPU 同类图见补充材料。

顺序链比单条较长 Picard 链快 4.43–6.92 倍，但轨迹长度和初始化不同，不能据此比较共同精度。各模型只有两份机制输入，操作探针还会扰动执行；现有结果支持额外工作与减速相容，尚不能唯一分离 JVP、扫描和主机控制的因果贡献。

4.4 固定预算误差与探索诊断

历史 Mac 较长预算 2048 步时，MALA 的两个执行器均在两个目标上取得最大函数误差区间低于 0.01；RWM 亦为两个，NUTS 为五个。同核配对误差在原表显示精度下相同，正文合并解释，执行费用分别报告。L1/L2 使用有限参考，L2 的全函数判定仍未定；完整数值、区间和全部目标在补充材料保留。

NUTS 的 2358 次发散和 6861 次积分上限命中均来自中心化漏斗 H1；双峰 M1 没有发散，但两个预算的最大参数 $\hat{R}$ 约 1.73，函数误差仍高。Windows 首轮则有 480/512 次拟合至少一个有限 $\hat{R} > 1.01$ ，108 次含常量变量或函数。数值核验成功、有限函数误差较小与整体探索充分并不等价。正在运行的紧凑研究使用独立调参和更长预算，将补足这一比较；本稿目前不报告其结果。

4.5 水井案例的推断任务

W1 使用 posteriordb 的 3020 个观测，模型为 $\Pr(y_i = 1) = \sigma(\alpha + \beta d_i/100)$ ，保留原模型对 α, β 的平坦先验。问题是距离与换井概率的关系，以及 0 米、100 米处预测概率的估计成本。两个不同距离处均出现异结局观测，足以证明该二维后验可正规化（补充材料附录）。Stan、NumPy、torch 密度及导数和 Windows 的 CPU/CUDA/R 数组路径已有核验；正式误差—成本结果随紧凑研究接收。稀有符号事件和非认证求积误差仍分别呈现。该案例已参与开发，不能称未见模型族测试。

5 讨论与结论

现有证据表明，时间并行能否降低成本取决于批处理节省的时间能否抵消重复映射、代理导数、扫描与同步。Mac 负结果和 Windows Picard 的局部执行收益都可由同一问题理解：并行宽度增加了可同时计算的位置，却不保证增加有用的路径推进。全拒绝例进一步说明，容易求解的轨迹可能没有推断价值。

这些结果不构成 CPU 不适合时间并行的普遍判断。Picard 原文在 Apple M3 上对由时滞微分方程定义的昂贵黑箱似然报告约 2.52 倍墙钟收益 [2]，成本结构不同于本文多数廉价目标。本文适配的窗口与停止规则也不同；精确恢复的因果性质不能消除浮点容差、初值和后验混合的影响。

本版软件使研究者能够定义目标、共享实际随机输入、检查指定执行器并保留失败，但尚非任意 Stan 模型的时间并行入口。Pyro NUTS 是研究 CLI，最外层 R 完整

费用仍待有限测量。比较也没有覆盖 MEADS、LAPS 等全部现代多链策略 [13, 14, 15]。现阶段最适合的用途是核验与比较已明确实现的目标和执行方式。

最终推断结论须依据紧凑研究的逐函数误差、诊断、失败与同一集合上的成本形成。预算不足时报告不足，而不按是否出现正加速决定追加配置。当前贡献是可重复使用的研究实现、配对执行证据及其机制解释；新算法、通用自动选择和普遍推断加速不在已完成贡献中。

代码、数据与构建材料

源码、协议、分析摘要和论文输入位于<https://github.com/AlchemistXC/ParallelBayes>。历史 CPU 原始证据见 `cpu-review-v1` Release。Windows 原始证据在 2026 年 10 月 7 日核对时仍为草稿 Release；当前公开状态需在最终交付前再核实，不能由源码可克隆推定原数组公众可取。新正文与独立补充材料共享文献表；随附构建入口及校验清单。保存分析到图文的重建不等于本轮重跑全部原始实验。

项目已有 Mac 与 Windows 安装及显式 R 集成证据；本稿中的外部目标示例使用公开接口。私有技能、第三方受限全文和本地环境不随公共材料发布。生成式工具辅助了代码、分析组织、图表与中文写作，相关陈述由实际记录及独立核验支撑；作者、单位、最终公开范围与投稿仍由研究者确认。

参考文献

- [1] Zoltowski D.M., Wu S., Gonzalez X., et al. [Parallelizing MCMC Across the Sequence Length](#). *Advances in Neural Information Processing Systems*, 2025, 38.
- [2] Grazzi S., Zanella G. [Parallel computations for Metropolis Markov chains with Picard maps](#). *Biometrika*, 2026, 113(2): asag022.
- [3] Carpenter B., Gelman A., Hoffman M.D., et al. [Stan: A Probabilistic Programming Language](#). *Journal of Statistical Software*, 2017, 76(1): 1–32.
- [4] Roualdes E.A., Ward B., Carpenter B., Seyboldt A., Axen S.D. [BridgeStan: Efficient in-memory access to the methods of a Stan model](#). *Journal of Open Source Software*, 2023, 8(87):5236.
- [5] Cabezas A., Corenflos A., Lao J., Louf R., et al. [BlackJAX: Composable Bayesian inference in JAX](#). *arXiv:2402.10797*, 2024.
- [6] Bingham E., Chen J.P., Jankowiak M., et al. [Pyro: Deep Universal Probabilistic Programming](#). *Journal of Machine Learning Research*, 2019, 20(28):1–6.
- [7] NIMBLE development team. [compareMCMCs: Tools for comparing MCMC efficiency from NIMBLE and other engines](#). 软件作者仓库，访问于 2026 年 10 月 3 日。

- [8] Kulkarni S., Shah K.D., Arora N., et al. [PPL Bench: Evaluation Framework For Probabilistic Programming Languages](#). arXiv:2010.08886v1, 2020. 软件来源: <https://github.com/facebookresearch/pplbench>。
- [9] Magnusson M., Torgander J., Bürkner P.-C., Zhang L., Carpenter B., Vehtari A. [posteriordb: Testing, Benchmarking and Developing Bayesian Inference Algorithms](#). Proceedings of Machine Learning Research, 2025, 258:1198–1206.
- [10] Stan development team. [posterior: Rhat, bulk ESS and tail ESS documentation](#). 软件文档, 版本 1.7.0。
- [11] Senne R. and contributors. [ParallelMCMC.jl](#). 软件仓库, 提交 7be9c86, 访问于 2026 年 10 月 6 日。
- [12] Sosa S. [BayesForge: Bayesian Inference using numpyro and XLA](#). R 包 0.0.1 参考手册, 访问于 2026 年 10 月 6 日。
- [13] Hoffman M.D., Sountsov P. [Tuning-Free Generalized Hamiltonian Monte Carlo](#). Proceedings of Machine Learning Research, 2022, 151:7799–7813.
- [14] Robnik J., Seljak U. [Faster Parallel MCMC: Metropolis Adjustment Is Best Served Warm](#). Proceedings of Machine Learning Research, 2026, 300:2719–2727.
- [15] Dance H., Glaser P., Orbanz P., Adams R.P. [Efficiently Vectorized MCMC on Modern Accelerators](#). Proceedings of Machine Learning Research, 2025, 267:12436–12458.
- [16] Gonzalez X., Warrington A., Smith J.T.H., Linderman S.W. [Towards Scalable and Stable Parallelization of Nonlinear RNNs](#). arXiv:2407.19115v3, 2025 (初版 2024) .
- [17] Vehtari A., Gelman A., Simpson D., Carpenter B., Bürkner P.-C. [Rank-Normalization, Folding, and Localization: An Improved R-hat for Assessing Convergence of MCMC](#). Bayesian Analysis, 2021, 16(2):667–718.
- [18] Modrák M., Moon A.H., Kim S., et al. [Simulation-Based Calibration Checking for Bayesian Computation: The Choice of Test Quantities Shapes Sensitivity](#). Bayesian Analysis. DOI:10.1214/23-BA1404.